S-ATTP for Numeric Classification Data: A Cluster-Stratified Partitioning Procedure and Its Validation Limits

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

***Abstract*—Train–test partitioning is part of the estimand in predictive evaluation: a random split estimates performance under a sampling design, not under every future population. This methods paper describes the currently inspected open-source Statistical Adaptive Train–Test Partitioner (S-ATTP) implementation. It maps finite numeric features into supervised linear-discriminant components, clusters the projected observations, forms joint class–cluster strata, and evaluates candidate stratified train fractions with a user-specified classifier. The candidate with the highest test score is returned. This paper derives the algorithm and documents constraints material to statistical use: LDA and clustering are fitted before candidate splitting, candidate test scores drive selection, singleton strata are assigned to training, and the implementation therefore does not provide an unbiased final test estimate. A nested or untouched outer evaluation set is required for generalization claims. This is an implementation-focused methods draft; no performance superiority is claimed.**

***Index Terms*—data partitioning, train–test split, linear discriminant analysis, K-means, validation leakage, generalization**

I. Introduction

The reported performance of a classifier depends on how evaluation data are selected. A single holdout can have substantial sampling variability, while repeated use of that holdout for model or split selection induces optimism [1], [5]. Stratification on labels preserves class proportions but need not represent local structure in the feature space. The S-ATTP implementation extends stratification to joint class–cluster strata after a supervised linear projection.

This contribution is a reproducible description and critical assessment of the S-ATTP implementation in the ATTP package [8]. It should not be read as a universal split guarantee or proof that the method improves generalization. The separate ATTP architecture reference describes additional classifier, regression, temporal, active-learning, and clustering variants; only the S-ATTP classifier partitioner was verified in the supplied package source. Broader architecture claims are therefore out of scope here.

II. Background and Motivation

Fisher's linear discriminant analysis (LDA) seeks directions with large between-class separation relative to within-class variation [2], [7]. K-means assigns projected points to centroids to minimize within-cluster squared distance [3], [6]. Stratified sampling then allocates observations across combinations of class and cluster. These are established methods; S-ATTP's design is their combination for candidate split selection.

An independent test set must not influence feature transformations, partition selection, preprocessing choices, or model selection if it is to estimate performance of the selected procedure. Leakage can occur even without direct target labels entering the classifier, whenever test observations inform learned representations or repeated selection [4]. This consideration is central to interpreting the current implementation.

III. Aim and Objectives

The aim is to specify how S-ATTP constructs and ranks candidate train–test partitions for finite numeric classification data. The objectives are to derive the LDA and clustering stages, state the exact class–cluster allocation and score-based choice, expose its leakage and selection limitations, and provide an outer-evaluation protocol suitable for empirical research.

IV. Method

Let $X \in \mathbb{R}^{n\times p}$ be a finite numeric feature matrix and $y_i \in \{1,\dots,C\}$ the class label. The current library validates nonempty numeric X, finite values, aligned nonmissing labels, and at least two classes. Data cleaning, categorical encoding, and scaling are external tasks and must be specified by the researcher.

A. Supervised dimension reduction

For class c, let $\bar{x}_c$ be the class mean, $\bar{x}$ the global mean, n_c the class size, and

$$S_W = \sum_{c=1}^{C} \sum_{i:y_i=c} (x_i - \bar{x}_c)(x_i - \bar{x}_c)^\top,$$

$$S_B = \sum_{c=1}^{C} n_c (\bar{x}_c - \bar{x})(\bar{x}_c - \bar{x})^\top.$$

Classical LDA seeks w maximizing Fisher's quotient

$$J(w) = \frac{w^\top S_B w}{w^\top S_W w},$$

equivalently a generalized eigenvector direction of $S_B w = \lambda S_W w$ when the solution is well-defined. With C classes, there are at most $\min(p, C-1)$ discriminant dimensions. The implementation fits scikit-learn LDA on all rows supplied to `sattp_split`, tries component counts within user bounds, and stops once the cumulative explained-discriminant variance reaches a configured threshold (default 0.85). The chosen projection is $Z = XW$.

B. Clustering and joint strata

For each candidate k, K-means minimizes

$$J_k = \sum_{j=1}^{k} \sum_{i:z_i \in C_j} \|z_i - \mu_j\|_2^2, \quad \mu_j = \frac{1}{|C_j|} \sum_{i:z_i \in C_j} z_i.$$

The implementation searches integers from `min_clusters` to `max_clusters`; it uses MiniBatchKMeans above 50,000 rows. A stratum is $S_i = (y_i, c_i)$, where c_i is the assigned cluster. Thus a conventional proportional class allocation is refined by the partition of the learned LDA space.

C. Candidate allocation and selection

For each requested training fraction α and candidate k, observations in strata with at least two members are allocated by stratified random splitting. Every singleton-stratum observation is assigned to training, since it cannot occur in both partitions. A cloned classifier $h_{\alpha,k}$ is fit on the resulting training subset and scored on the candidate test subset:

$$(\alpha^\star, k^\star) = \arg \max_{(\alpha,k) \in \mathcal{A}} \widehat{Q}_{\text{test}}(h_{\alpha,k}),$$

where $\mathcal{A}$ is the set of feasible candidates and Q is the estimator's default score or user-supplied scoring function. The returned result includes selected observations, indices, scores, component information, and the candidate-score table. If no candidate is feasible, the function raises `NoValidSplitError`.

V. Validation design for generalization

The returned candidate test score is a tuning criterion, not an unbiased estimate of the performance of the selected classifier. LDA uses all supplied X, y before candidate splits are generated; K-means also sees all supplied projected observations; and the candidate test score determines which split is returned. Any preprocessing fitted before this call may introduce further leakage if it saw future test data. These choices can be useful for exploratory split construction, but they make the within-call test score unsuitable as final validation.

A defensible empirical comparison first creates an untouched outer test set or outer fold using the intended deployment unit (time, site, patient, entity, or random sampling as justified). Every preprocessing fit, ATTP candidate search, model choice, and resampling operation is confined to the outer-training data. The selected model is evaluated once on the untouched outer observations. For temporal or grouped deployment, the outer design must preserve temporal order or group separation; random cluster-stratification is not a substitute. Report variability across independent outer units, class prevalence, the number of singleton strata, and computational cost.

VI. Limitations and interpretation

LDA is supervised and linear; K-means favors spherical Euclidean clusters and is sensitive to scale, outliers, and initialization. The released function expects finite numeric input and performs no scaling, imputation, categorical encoding, or resampling. Cluster counts and training ratios are user-selected search ranges. Singleton strata are always placed in training, which can make the realized split depart from the nominal ratio or leave rare feature–class combinations absent from test. A candidate accuracy score can favor splits that are easier for the chosen classifier rather than splits that are representative of a target population.

S-ATTP does not detect or guarantee an out-of-distribution validation set. A separate outer distribution-shift evaluation must define the target population and isolate it before model development. Nor does non-significance of a distribution test demonstrate parity. No algorithm guarantees robust generalization across unseen domains.

VII. Conclusion

S-ATTP implements a class–cluster stratified candidate split search on LDA-reduced numeric data. Its design can be studied as a partitioning heuristic, but its current fit-before-split and score-based selection prevent treating its internal test score as a final generalization estimate. Future empirical work should compare the method with prespecified splitters in nested or untouched outer evaluations and report uncertainty, coverage, class composition, leakage controls, and runtime. The supplied implementation and tests support an implementation description, not a claim of superiority.

References

[1] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Statistics Surveys*, vol. 4, pp. 40–79, 2010.

[2] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.

[3] A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010.

[4] S. Kaufman, S. Rosset, and C. Perlich, "Leakage in data mining: Formulation, detection, and avoidance," in *Proc. 18th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2012, pp. 556–563.

[5] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artificial Intelligence*, 1995, pp. 1137–1143.
[6] S. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
[7] G. McLachlan, *Discriminant Analysis and Statistical Pattern Recognition*. Wiley, 2004.
[8] J. Monchwe, *ATTP*, software. [Online]. Available: https://github.com/Jabualani/ATTP.